



MoonChunk

Dokumentacja języka

Domenowy język (DSL) do deklaratywno-imperatywnego opisywania procesu generowania statycznych stron HTML — z jawnym modelem wykonania, kontrolą typów i przyjazną diagnostyką błędów.

Wersja 1.0

Projekt MoonChunk

Typ Dokumentacja użytkownika

Język Polski

— OFICJALNA DOKUMENTACJA TECHNICZNA —

01	Wprowadzenie	12	Strony, content i renderowanie HTML
02	Filozofia i przeznaczenie języka	13	Metadane i layout wewnętrzny
03	Szybki start	14	Wbudowane funkcje runtime
04	Model programu w MoonChunk	15	Diagnostyka błędów
05	Podstawowa składnia	16	Przykłady kompletne
06	Typy i system typów	17	Dobre praktyki
07	Zmienne, mutowalność i scope	18	Najczęstsze pułapki i jak ich unikać
08	Wyrażenia i operatory	19	Architektura implementacji (ANTLR + runtime)
09	Instrukcje sterujące	20	Słownik pojęć
10	Funkcje	21	Podsumowanie
11	Importy, moduły i include		



ROZDZIAŁ 01

Wprowadzenie

MoonChunk to język domenowy (DSL), którego celem jest deklaratywno-imperatywne opisywanie procesu generowania statycznych stron HTML. Język został zaprojektowany tak, aby łączyć:

- czytelność składni dla człowieka,
- jawny model wykonania,
- kontrolę typów,
- oraz przyjazną diagnostykę błędów.

Programista pracuje na plikach `.mncnk`, które definiują:

- chunki (moduły logiczne),
- strony (`page`),
- zmienne i metadane,
- kontrolę przepływu (`if`, `for`, `while`),
- funkcje (w tym rekurencję),
- oraz końcowy punkt wykonania `moon(...)`.

WYNIK WYKONANIA

Efektem uruchomienia programu są wygenerowane pliki HTML w katalogu wyjściowym.



ROZDZIAŁ 02

Filozofia i przeznaczenie języka

MoonChunk **nie jest zamiennikiem** pełnego języka ogólnego przeznaczenia. Został zaprojektowany pod konkretny przypadek użycia: **generowanie stron statycznych z logiką kompozycji, typowania i kontroli przepływu**.

Najważniejsze założenia

1. **Przewidywalność wykonania** — kod wykonuje się w jasno określonym porządku i scope.
2. **Jawność punktu wejścia** — nic nie dzieje się „magicznie” bez `moon(...)`.
3. **Bezpieczeństwo typów** — runtime waliduje zgodność typów i zgłasza błędy z pozycją.
4. **Przyjazność dla autora treści i logiki** — można budować dynamiczny HTML bez rozbudowanego frameworka.
5. **Modularność** — `import` + `@include` pozwalają kompozycję wieloplikową.



ROZDZIAŁ 03

Szybki start

3.1. Instalacja zależności

```
yarn install
```

3.2. Budowanie projektu

```
yarn build
```

3.3. Uruchamianie programu `.mncnk`

```
yarn start examples/scenarios/18-recursive-function/site.mncnk
```

3.4. Tryb debug

```
yarn start:debug examples/scenarios/18-recursive-function/site.mncnk
```

3.5. Kontrola jakości

```
yarn lint  
yarn run check
```



ROZDZIAŁ 04

Model programu w MoonChunk

Program składa się z:

- opcjonalnych `import`,
- deklaracji `chunk`,
- oraz wywołań `moon(...)`.

Najprostsza forma

```
chunk "Main" {  
  output: "./dist";  
  page "" {  
    content {  
      <h1>Hello</h1>  
    };  
  };  
};  
  
moon(Main);
```

KLUCZOWE ZASADY

- `moon(...)` uruchamia wskazany chunk.
- Bez `moon(...)` kod nie zostanie wykonany jako entrypoint.
- Importowany chunk musi być `export`.



ROZDZIAŁ 05

Podstawowa składnia

5.1. Instrukcje i średniki

W MoonChunk instrukcje kończą się średnikiem `;`.

```
let a: int = 1;
const title: string = "MoonChunk";
```

5.2. Bloki

Bloki używają `{ ... }`:

```
{
  let local: int = 10;
  print(local);
};
```

5.3. Komentarze

Komentarz liniowy:

```
// To jest komentarz
let a: int = 1;
```



ROZDZIAŁ 06

Typy i system typów

Obsługiwane typy podstawowe:

- `int`, `float`, `double`, `number`
- `bool`, `string`
- `array`, `object`
- `null`, `undefined`, `unknown`, `any`
- `void` (dla funkcji)

6.1. Deklaracja typowana

```
let a: int = 5;  
let b: double = 2.5;  
let title: string = "Demo";  
let ok: bool = true;
```

6.2. Promocja typów liczbowych

```
let x: float = 1;  
let y: double = 2;
```

6.3. Rzutowania

MoonChunk wspiera dwa style:

Styl `as`:

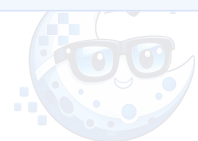
```
let a: int = 3;  
let b: double = a as double;
```

Styl `C`:

```
let c: int = (int)2.9;
```

6.4. Konwersje przez `unknown` / `any`

W kontrolowanych przypadkach można użyć mostu typów:




```
let raw: int = -3;  
print(raw as unknown as bool);
```



ROZDZIAŁ 07

Zmienne, mutowalność i scope

7.1. `let` i `const`

- `let` pozwala na przypisanie nowej wartości.
- `const` blokuje ponowne przypisanie.

```
let counter: int = 0;
counter = counter + 1;

const title: string = "MoonChunk";
// title = "Nowy"; // błąd
```

7.2. Redeclaracja

- W tym samym scope redeklaracja jest błędem.
- W dozwolonych scope potomnych możliwe jest cieniowanie zgodnie z aktualnymi regułami runtime.

7.3. Scope blokowy

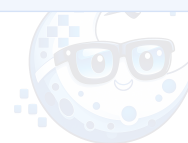
```
let a: int = 1;
{
  let a: int = 5;
  print(a); // 5
};
print(a); // 1
```

7.4. Dostęp do scope nadrzędnego

Można odwołać się explicite do rodzica:

```
let a: int = 1;
{
  let a: int = 5;
  print(parent::a); // 1
};
```

Dla głębszego poziomu:



```
print(parent::parent::a);
```

7.5. Globalne zmienne przez `env`

```
chunk "Main" {  
  env {  
    global siteName: string = "My Site";  
  };  
};
```



ROZDZIAŁ 08

Wyrażenia i operatory

8.1. Arytmetyka

`+`, `-`, `*`, `/`, `%`, oraz nawiasowanie.

```
let a: int = (2 + 3) * 4;  
let b: int = 7 % 3;
```

8.2. Logika

`and`, `or`, `not`.

```
let x: bool = true;  
let y: bool = false;  
let z: bool = x and not y;
```

8.3. Porównania

`==`, `!=`, `<`, `>`, `<=`, `>=`.

```
let ok: bool = 10 > 5;
```

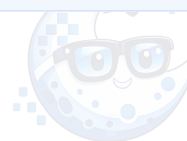
8.4. Operator warunkowy

```
let label: string = (true ? "A" : "B");
```

8.5. Inkrementacja

Obsługiwane: `i++` oraz `++i`.

```
let i: int = 1;  
print(i++); // 1  
print(i);   // 2  
print(++i); // 3
```



ROZDZIAŁ 09

Instrukcje sterujące

9.1. `if / else`

```
if (isLoggedIn == true) {  
    print("zalogowany");  
} else {  
    print("gość");  
};
```

9.2. `for`

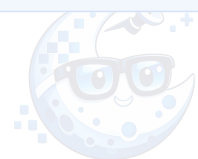
```
for(let i: int = 0; i < 10; i = i + 2) {  
    print(i);  
};
```

9.3. `while`

```
let i: int = 0;  
while (i < 3) {  
    print(i);  
    i = i + 1;  
};
```

9.4. `break` i `continue`

```
for(let i: int = 0; i < 10; i++) {  
    if (i == 3) {  
        continue;  
    };  
    if (i == 7) {  
        break;  
    };  
    print(i);  
};
```



ROZDZIAŁ 10

Funkcje

10.1. Funkcja klasyczna

```
function add(a: int, b: int): int {  
    return a + b;  
}
```

10.2. Funkcja strzałkowa

```
sum(a: int, b: int): int => a + b;
```

10.3. Rekurencja

```
function fact(n: int): int {  
    if (n <= 1) {  
        return 1;  
    };  
    return n * fact(n - 1);  
}  
  
print(fact(5));
```

10.4. void

```
function logMessage(text: string): void {  
    print(text);  
    return;  
}
```



ROZDZIAŁ 11

Importy, moduły i include

MoonChunk wspiera dwa style importu.

11.1. Import nazwany

```
import { HeroChunk } from "./hero.mncnk";
```

Użycie:

```
chunk "Main" {  
    @include HeroChunk;  
};
```

11.2. Import namespace

```
import * as Fragments from "./fragments.mncnk";
```

Użycie:

```
chunk "Main" {  
    @include Fragments.HeroChunk;  
};
```

11.3. Wymaganie `export`

Chunk importowany musi być eksportowany:

```
export chunk "HeroChunk" {  
    title: "Hero";  
};
```



ROZDZIAŁ 12

Strony, content i renderowanie HTML

12.1. Definicja strony

```
page "about" {  
  content {  
    <h1>O nas</h1>  
    <p>Treść strony.</p>  
  };  
};
```

12.2. Interpolacja wartości

W `content` używamy `{ ... }`:

```
let title: string = "Aktualności";  
content {  
  <h1>{title}</h1>  
};
```

12.3. Content w pętlach i warunkach

```
for(let i: int = 0; i < 3; i++) {  
  content {  
    <p>Element: {i}</p>  
  };  
};
```



ROZDZIAŁ 13

Metadane i layout wewnętrzny

MoonChunk posiada wewnętrzny `base.tpl`. Metadane ustawia się bezpośrednio w pliku `.mncnk`.

```
title: "Moja strona";
metaDescription: "Opis strony";
lang: "pl";
```

Wspierane są między innymi:

- podstawowe meta (`title` , `metaDescription` , `metaKeywords` , `metaAuthor` , `metaRobots`),
- Open Graph (`ogTitle` , `ogDescription` , `ogImage` , ...),
- Twitter Card (`twitterTitle` , `twitterDescription` , ...),
- oraz pola layoutowe (`header` , `footer` , `scripts` , `styles` , ...).

OPTIMALIZACJA

Puste metadane nie są renderowane jako niepotrzebne tagi.



ROZDZIAŁ 14

Wbudowane funkcje runtime

14.1. `print(...)`

Służy do diagnostyki i podglądu wartości:

```
print("counter:", 10);
```

14.2. `data(...json)`

Wczytanie danych z pliku JSON:

```
let posts: array = data("posts.json");
```



ROZDZIAŁ 15

Diagnostyka błędów

MoonChunk kładzie nacisk na czytelne błędy:

- błędy leksykalne,
- błędy składniowe,
- błędy runtime i typów,
- informacja o linii i kolumnie.

15.1. Przykład błędu ze wskazaniem poprawki

```
let counter: int = 1;  
print(countr);
```

Komunikat może zawierać sugestię:

SUGESTIA KOMPILATORA

Did you mean 'counter'?

15.2. Przykład błędu typu

```
let a: bool = true;  
a = 3;
```

Runtime zgłosi niezgodność typu.

15.3. Przykład błędu sterowania

`break` lub `continue` poza pętlą zgłaszają błąd semantyczny.



ROZDZIAŁ 16

Przykłady kompletne

16.1. Mini strona z pętlą

```
chunk "Main" {
  output: "./dist";
  title: "Blog";

  let posts: array = [
    { title: "A", views: 10 },
    { title: "B", views: 20 }
  ];

  page "" {
    content {
      <h1>{title}</h1>
    };

    for(let i: int = 0; i < 2; i++) {
      content {
        <article>
          <h2>{posts[i].title}</h2>
          <p>Views: {posts[i].views}</p>
        </article>
      };
    };
  };
};

moon(Main);
```



16.2. Scope i parent

```
chunk "ScopeDemo" {  
  output: "./dist";  
  let value: int = 1;  
  
  {  
    let value: int = 5;  
    print(value);           // 5  
    print(parent::value);  // 1  
  };  
};  
  
moon(ScopeDemo);
```

16.3. Funkcja rekurencyjna

```
chunk "RecDemo" {  
  output: "./dist";  
  
  function fib(n: int): int {  
    if (n <= 1) {  
      return n;  
    };  
    return fib(n - 1) + fib(n - 2);  
  }  
  
  print(fib(6));  
};  
  
moon(RecDemo);
```



ROZDZIAŁ 17

Dobre praktyki

1. Trzymaj logikę podzieloną na chunki.

Mniejszy chunk = łatwiejsze testowanie.

2. Używaj `moon(...)` jawnie i świadomie.

Unikniesz nieoczekiwanego uruchamiania.

3. Waliduj scenariusze `error-*`.

To stabilizuje diagnostykę.

4. Nie mieszaj odpowiedzialności.

Dane w `data(...)`, render w `content`, logika w funkcjach.

5. Zachowuj spójność typów.

Mniej rzutowań = mniej ryzyka.



ROZDZIAŁ 18

Najczęstsze pułapki i jak ich unikać

18.1. Brak `moon(...)`

Objaw: brak wygenerowanych plików mimo poprawnego kodu.

Rozwiązanie: dodaj entypoint.

18.2. Redeklaracja w tym samym scope

Objaw: błąd „Variable redeclaration...”.

Rozwiązanie: zmień nazwę lub użyj przypisania.

18.3. Błędny import

Objaw: chunk nie znaleziony / nieeksportowany.

Rozwiązanie: sprawdź `export chunk` i ścieżkę pliku.

18.4. Operacje typowe dla liczb na `string`

Objaw: błąd typu liczbowego.

Rozwiązanie: konwersja lub poprawa deklaracji.



ROZDZIAŁ 19

Architektura implementacji (ANTLR + runtime)

Projekt technicznie dzieli się na pięć warstw:

1. Lexer i Parser (ANTLR4TS)

- `MoonChunkLexer.g4`
- `MoonChunkParser.g4`

2. Budowa AST

- odwiedzanie parse tree i mapowanie na struktury runtime.

3. Runtime Executor

- wykonanie instrukcji,
- zarządzanie scope,
- obsługa typów i funkcji.

4. Renderer HTML

- przetwarzanie `content`,
- interpolacje,
- metadane i finalny dokument.

5. Warstwa CLI / DX

- skrypty `start`, `start:debug`, `build`, `check`,
- normalizacja i formatowanie błędów.



ROZDZIAŁ 20

Słownik pojęć

- **DSL** — język dedykowany do konkretnej domeny.
- **Chunk** — moduł logiki i konfiguracji w MoonChunk.
- **Entrypoint** — punkt startu wykonania (`moon( ... )`).
- **Scope** — zasięg widoczności zmiennych.
- **AST** — pośrednia reprezentacja składni programu.
- **Runtime** — silnik wykonujący AST.



ROZDZIAŁ 21

Podsumowanie

MoonChunk to praktyczny, typowany język DSL do generowania HTML, który oferuje:

- modularność (`import` , `@include`),
- jawne uruchamianie (`moon(...)`),
- kontrolę przepływu i funkcje (także rekurencję),
- scope blokowe z dostępem `parent::` ,
- oraz przyjazną diagnostykę błędów.

W efekcie dostajemy narzędzie, które jest:

- wystarczająco ekspresyjne do realnych scenariuszy,
- a jednocześnie przewidywalne i łatwe w utrzymaniu.

WNIOSKI

To dobry fundament do dalszego rozwoju języka i ekosystemu narzędzi.





Dziękujemy

za zapoznanie się z dokumentacją MoonChunk

MoonChunk to narzędzie zaprojektowane z myślą o przewidywalności, bezpieczeństwie typów i ergonomii autora. Mamy nadzieję, że ta dokumentacja pomoże Ci w pełni wykorzystać jego możliwości — od pierwszego chunk "Main" aż po zaawansowane scenariusze rekurencji i kompozycji modułów.

Powodzenia w budowaniu stron z MoonChunk.

MOONCHUNK · DOKUMENTACJA V1.0 · 2026